Project code description

```
def load_tasks():
    if os.path.exists(FILE_NAME):
        try:
            with open(FILE_NAME, "r") as file:
                return json.load(file)
        except json.JSONDecodeError:
            return []
    return []
```

Check Existence: It first checks if the file (defined by FILE_NAME) actually exists on your computer using os.path.exists.

Open and Read: If the file is there, it opens it in read mode ("r").

Parse JSON: It attempts to convert the text inside that file into a Python object (like a list or dictionary) using json.load(file).

Error Handling:

Corrupt Data: If the file exists but contains invalid JSON (e.g., it's empty or has a typo), the except json.JSONDecodeError block catches the crash and returns an empty list [] instead.

Missing File: If the file doesn't exist at all, it skips the logic and returns an empty list [].

```python
def save_tasks(tasks):
    with open(FILE_NAME, "w") as file:
        json.dump(tasks, file, indent=4)
```

Open in Write Mode: open(FILE_NAME, "w") opens the specified file. The "w" (write) mode is important because it either creates the file if it doesn't exist or completely overwrites it if it does.

Serialize Data: json.dump(tasks, file) takes your Python object (the tasks list) and converts it into a JSON-formatted string inside that file.

Make it Readable: The indent=4 argument is optional but helpful; it adds spaces and line breaks to the file so a human can easily read the JSON if they open it in a text editor.

Automatic Close: Using the with statement ensures the file is properly saved and closed as soon as the indented code block finishes.

```python
def add_task(tasks):
    print("\n--- Add New Task ---")
    task_name = input("Enter task name: ").strip()
    subject = input("Enter subject name: ").strip()
    deadline = input("Enter deadline (YYYY-MM-DD): ").strip()
    priority = input("Enter priority (High/Medium/Low): ").strip().capitalize()

    if not task_name or not subject or not deadline:
        print("Task name, subject, and deadline cannot be empty.")
        return

    try:
        datetime.strptime(deadline, "%Y-%m-%d")
    except ValueError:
        print("Invalid date format. Please use YYYY-MM-DD.")
        return

    if priority not in ["High", "Medium", "Low"]:
        print("Invalid priority. Setting priority to Medium.")
        priority = "Medium"

    task = {
        "task_name": task_name,
        "subject": subject,
        "deadline": deadline,
        "priority": priority,
        "status": "Pending"
    }

    tasks.append(task)
    save_tasks(tasks)
    print("Task added successfully.")
```

User Input: It uses input().strip() to collect the task name, subject, deadline, and priority. The .strip() removes any accidental leading or trailing spaces.

Basic Validation: It checks if the name, subject, or deadline are empty. If they are, it stops the function early with a return to prevent saving "ghost" tasks.

Date Validation: It uses datetime.strptime to ensure the deadline matches the YYYY-MM-DD format. If the user types something else (like "tomorrow"), the except block catches the error and stops the function.

Priority Safeguard: If the user enters a priority other than High, Medium, or Low, the code doesn't crash; it simply defaults the task to "Medium".

Dictionary Creation: It bundles all those individual pieces of info into a single Python dictionary called task and adds a default "status": "Pending".

Updating & Saving:

tasks.append(task) adds this new dictionary to your main list.

save_tasks(tasks) calls your previous method to immediately write the updated list to your JSON file

```python
def view_tasks(tasks):
    print("\n--- All Tasks ---")
    if not tasks:
        print("No tasks available.")
        return

    for i, task in enumerate(tasks, start=1):
        print(f"\nTask {i}")
        print(f"Task Name : {task['task_name']}")
        print(f"Subject   : {task['subject']}")
        print(f"Deadline  : {task['deadline']}")
        print(f"Priority  : {task['priority']}")
        print(f"Status    : {task['status']}")
```

def view_tasks(tasks):

Defines the function. it expects a list called tasks (which contains task dictionaries) as an input.

print("\n--- All Tasks ---")

Prints a header for the user. The \n adds a blank line at the top to keep the output clean.

if not tasks:

Checks if the list is empty. In Python, an empty list evaluates to False, so not tasks becomes True if there are no items.

print("No tasks available.")

If the list is empty, it informs the user so they aren't staring at a blank screen.

return

Exits the function immediately. There is no point in trying to loop through a list that doesn't exist.

for i, task in enumerate(tasks, start=1):

This is the most "Pythonic" line.

enumerate looks at each item in the list and gives it a counter.

start=1 tells the counter to begin at 1 (instead of the default 0) so it's more natural for users.

i gets the number (1, 2, 3...) and task gets the actual dictionary data.

print(f"\nTask {i}")

Uses an f-string to print the task number (e.g., "Task 1").

print(f"Task Name : {task['task_name']}") (and subsequent lines)

Accesses the specific values stored in the dictionary using their keys ('task_name', 'subject', etc.) and prints them in a readable format.

```
def mark_task_completed(tasks):
    view_tasks(tasks)
    if not tasks:
        return

    try:
        task_number = int(input("\nEnter task number to mark
as completed: "))
        if 1 <= task_number <= len(tasks):
            tasks[task_number - 1]["status"] = "Completed"
            save_tasks(tasks)
            print("Task marked as completed.")
        else:
            print("Invalid task number.")
    except ValueError:
        print("Please enter a valid number.")
```

Display First: It calls view_tasks(tasks) so the user can actually see the list and know which number to type.

Safety Check: If the list is empty (if not tasks), it immediately exits (return) because there's nothing to mark.

Input Conversion: It asks for a task number and uses int() to turn the user's text into a number.

The "Off-by-One" Logic:

Since humans start counting at 1 but Python lists start at 0, the code uses tasks [task_number - 1].

It checks if the number is valid (between 1 and the total length of the list) before trying to access it.

Status Update: It targets the "status" key in that specific task dictionary and changes its value to "Completed".

Saving Changes: It calls your save_tasks(tasks) method to make the change permanent in your JSON file.

Error Handling: The except ValueError block is there to catch "garbage" input. If a user types "five" instead of "5", the program prints an error message instead of crashing.

```python
def delete_task(tasks):
    view_tasks(tasks)
    if not tasks:
        return

    try:
        task_number = int(input("\nEnter task number to delete: "))
        if 1 <= task_number <= len(tasks):
            deleted_task = tasks.pop(task_number - 1)
            save_tasks(tasks)
            print(f"Task '{deleted_task['task_name']}' deleted
successfully.")
        else:
            print("Invalid task number.")
    except ValueError:
        print("Please enter a valid number.")
```

Visual Aid: It calls view_tasks(tasks) first so the user knows which number corresponds to the task they want to remove.

Safety Exit: If there are no tasks in the list (if not tasks), the function stops immediately to avoid errors.

The pop() Method: This is the most important part. tasks.pop(task_number - 1) does two things at once:

It removes the item at that specific index from the list.

It returns that item, which the code stores in deleted_task so it can mention the name in the final confirmation message.

Index Adjustment: Like the "mark completed" method, it subtracts 1 from the user's input because Python lists start counting at 0.

Data Persistence: It calls save_tasks(tasks) to ensure the task is gone from the JSON file, not just from the computer's temporary memory.

Error Handling:
Boundary Check: The if statement ensures the user didn't pick a number higher than the list length.
Input Check: The try/except block catches cases where a user types text instead of a number.

```python
def search_by_subject(tasks):
    print("\n--- Search Tasks by Subject ---")
    subject_name = input("Enter subject name to search:
").strip().lower()

    found = False
    for i, task in enumerate(tasks, start=1):
        if task["subject"].lower() == subject_name:
            found = True
            print(f"\nTask {i}")
            print(f"Task Name : {task['task_name']}")
            print(f"Subject   : {task['subject']}")
            print(f"Deadline  : {task['deadline']}")
            print(f"Priority  : {task['priority']}")
            print(f"Status    : {task['status']}")

    if not found:
        print("No tasks found for this subject.")
```

Normalization: It takes the user's input and uses .lower() on both the search term and the stored subject name. This makes the search case-insensitive (e.g., "Math" will match "math").

The "Found" Flag: It initializes found = False. This acts as a simple switch to keep track of whether any matches were discovered during the search.

Looping with enumerate:
It loops through the entire tasks list.
enumerate(tasks, start=1) allows the code to keep track of the task's original position (i) starting from 1, so the user knows which task number it is.

The Match Check: Inside the loop, it compares the subject key of each task to the user's input. If they match, it sets found = True and prints out all the details for that specific task.

The "No Results" Message: After the loop finishes, it checks the found flag. If it is still False, it tells the user that nothing was found.

```python
def show_overdue_tasks(tasks):
    print("\n--- Overdue Tasks ---")
    today = datetime.today().date()
    found = False

    for i, task in enumerate(tasks, start=1):
        deadline_date = datetime.strptime(task["deadline"], "%Y-%m-
%d").date()
        if task["status"] == "Pending" and deadline_date < today:
            found = True
            print(f"\nTask {i}")
            print(f"Task Name : {task['task_name']}")
            print(f"Subject   : {task['subject']}")
            print(f"Deadline  : {task['deadline']}")
            print(f"Priority  : {task['priority']}")
            print(f"Status    : {task['status']}")

    if not found:
        print("No overdue tasks found.")
```

Get Current Date: datetime.today().date() gets today's date (ignoring the specific time) so it can be compared against task deadlines.

Date Conversion: Inside the loop, datetime.strptime(task["deadline"], "%Y-%m-%d").date() converts the string stored in your JSON file (like "2023-12-01") back into a date object that Python can mathematically compare.

The "Overdue" Logic: It checks for two specific conditions at once:

1. task["status"] == "Pending": The task isn't finished yet.
2. deadline_date < today: The deadline is earlier than today's date.

The "Found" Flag: Similar to your search function, it uses found = False to track if any tasks match these criteria. If none do, it tells the user "No overdue tasks found" at the very end.

Displaying Results: For every match it finds, it prints out the full details (Name, Subject, Priority, etc.) and uses enumerate(tasks, start=1) to give each task its correct list number.

```python
def main():
    tasks = load_tasks()

    while True:
        print("\n========== Personal Study Planner ==========")
        print("1. Add Task")
        print("2. View All Tasks")
        print("3. Mark Task as Completed")
        print("4. Delete Task")
        print("5. Search Tasks by Subject")
        print("6. Show Overdue Tasks")
        print("7. Exit")

        choice = input("Enter your choice: ").strip()

        if choice == "1":
            add_task(tasks)
        elif choice == "2":
            view_tasks(tasks)
        elif choice == "3":
            mark_task_completed(tasks)
        elif choice == "4":
            delete_task(tasks)
        elif choice == "5":
            search_by_subject(tasks)
        elif choice == "6":
            show_overdue_tasks(tasks)
        elif choice == "7":
            print("Thank you for using Personal Study Planner.")
            break
        else:
            print("Invalid choice. Please try again.")


if __name__ == "__main__":
    main()
```

Initialization: It starts by calling load_tasks(). This ensures that as soon as the program opens, your saved data is pulled from the JSON file into the tasks list.

The Infinite Loop (while True): This keeps the program running. Without this, the program would run once and immediately close. It allows you to perform multiple actions (like adding then viewing) in one session.

The Menu Interface: It prints a simple text-based menu so the user knows which numbers correspond to which features.

Input & Routing (if/elif/else):

1. It captures the user's choice.
2. It matches that choice to the correct function (e.g., if you type "1", it runs add_task(tasks)).
3. It passes the tasks list into every function so they all work on the same data.

Graceful Exit: If the user types "7", the break command stops the while loop, and the program finishes.

The "Safety Guard" (if __name__ == "__main__":): This is a Python best practice. It ensures that main() only runs if you execute this specific file directly, preventing it from running accidentally if you ever import this code into another script.